

Research Statement

Atharv Kulkarni

✉ athkulk@unc.edu

🌐 [LinkedIn](#)

🔍 [Google Scholar](#)

🐙 [GitHub](#)

Research Vision

My research focuses on how language models can reason and learn reliably when fluent generation is not enough. I am interested in the choices that sit around the model itself: how a problem is represented, what feedback the model receives, how competing reasoning strategies are evaluated, and what experience is used for post-training. Across my work, I have studied these questions through neurosymbolic reasoning, reinforcement learning from verifiable feedback, code generation and repair, and long-horizon agents that interact with tools and stateful environments.

A common lesson has emerged from these projects. Better reasoning does not come only from asking a model to think longer or from scaling the model. It also depends on giving the model structure that constrains the problem, feedback that reflects real outcomes, and training data that exposes the limits of its current behavior. My work has moved across these layers, from designing symbolic representations for robust reasoning, to learning from environmental feedback, to choosing among alternative reasoning strategies, and most recently to generating harder agentic experience at scale.

Structuring Reasoning Beyond Language

I began this line of work at the University of Pennsylvania with Prof. Dan Roth, Prof. Vivek Gupta and Prof. Vivek Srikumar. It became my first-author paper, *LLM-Symbolic Integration for Robust Temporal Tabular Reasoning*, published in Findings of ACL 2025 [1]. We started with a basic question: when an LLM answers a question over structured data correctly, did it actually reason over the evidence, or did it recognize facts and patterns already present in pre-training?

To separate these possibilities, I developed TEMPTABQA-C, a controlled benchmark temporal question-answering on tabular data. The benchmark varies the underlying facts, table size, and reasoning complexity. Its counterfactual split is especially useful because the same reasoning problem can be evaluated after factual values are changed. A model that follows the evidence should adapt to the new values. A model that relies on memorized knowledge should not. I built the data-processing and relational-database pipelines used to generate and evaluate these controlled examples.

The results showed that the problem was not solved by stronger prompting alone. Direct prompting degraded when facts became counterfactual, when tables grew larger, and when questions required more reasoning steps. This suggested that part of the failure came from the representation itself. Raw tables mix the structure of the problem with many values that can distract the model or trigger memorized associations. We therefore asked the model to reason from a database schema rather than from the table contents. The model generated an executable SQL program, and the database engine computed the answer.

This change made reasoning much more stable. For GPT-4o, the gap between original and counterfactual questions fell from 21.4 points with chain-of-thought prompting to 3.0 points with adaptive symbolic reasoning. The gap between small and large tables fell from 18.0 points to 1.1 points. On the hardest questions, the symbolic approach also surpassed chain of thought, reaching 66.7% exact match compared with 64.1%. The broader lesson was that the interface through which a model reasons can matter as much as the reasoning procedure. When a task has a formal structure, moving part of the reasoning into a symbolic representation can remove sources of error that language-only reasoning leaves unresolved.

Learning from Verifiable Feedback

The symbolic system improved reasoning at inference time, but it exposed a second limitation. The model still had to learn how to produce the right formal program from examples of programs. At the University of Utah, working with Prof. Vivek Srikumar, I asked whether that supervision was necessary. If the environment can tell us whether a generated program worked, can a model learn the program without ever seeing the gold program? This became *Reinforcing Code Generation: Improving Text-to-SQL with Execution-Based Learning* [2].

I framed program generation as reinforcement learning with a live database in the loop. The model receives a question and schema, proposes SQL, executes it, and learns from the result. Because exact correctness alone gives a sparse

training signal, I designed the reward to distinguish several outcomes. Invalid programs are penalized, executable programs receive a small reward, partially correct answers receive partial credit, and exact answers receive a larger reward. I trained the model with Group Relative Policy Optimization using question-answer pairs only, without gold SQL during training.

The model learned substantially better programs from this weak supervision. Exact-match accuracy increased from 31.49% to 49.83%, while syntax errors fell from 25.43% to 14.71%. The gains were largest on harder queries, where exact match increased from 13.27% to 27.53%. On counterfactual data, accuracy rose from 29.18% to 47.07%, bringing the 7B model close to a model an order of magnitude larger.

The failures were also informative. In one model family, the partial-credit reward encouraged a form of reward hacking: the model learned to return broad result sets that contained some correct answers rather than solving the intended query. This changed how I thought about verifiable feedback. An executable reward gives the model an objective grounded outside the model, but it does not remove the reward-design problem. The model optimizes the signal it receives, so the reward must distinguish real progress from shortcuts that only look successful under the metric.

Adaptive Reasoning and Verification

At UNC Chapel Hill, with Prof. Mohit Bansal and collaborators, I turned to a different question about model reasoning. When an LLM-generated program fails, should the model refine its existing solution, or should it discard that reasoning and start again? Most repair systems default to debugging. In our analysis, this assumption failed in a systematic way. For programs with severe errors, regeneration from the original problem outperformed debugging by 25.8 points. As the original program became more correct, that advantage shrank, and debugging eventually became the better strategy. This work became my first-author paper, *Debug or Regenerate? Test-Guided Selection for LLM Code Repair*, published in Findings of EMNLP 2026 [3].

The result suggested that code repair is not only a generation problem. It is also a strategy-selection problem. Debugging preserves useful structure but can remain trapped in flawed reasoning. Regeneration can escape that reasoning but may throw away parts that were already correct. We developed PATSSEL, Program-Adaptive Test-guided Repair Strategy Selection, to choose between these two behaviors for each failure. PATSSEL generates one debugged candidate and one regenerated candidate, probes both with generated executable tests, and selects the candidate with stronger behavioral evidence.

The verifier is important because hidden benchmark tests are not available at deployment time. We therefore trained a unit-test generator with GRPO to produce executable tests that agree with correct program behavior. We also trained a debugger with a separate reward based on improvement over the original program. This let us study reward design for different roles within the same system: one model learns to repair code, while another learns to produce evidence that can distinguish competing repairs.

Across LeetCodeDataset, LiveCodeBench, and HE+Fix, PATSSEL outperformed compute-matched single-strategy repair across model families. The average absolute gain over the strongest single-strategy baseline was 2.7 points with Qwen3-8B and 3.2 points with Llama3-8B. On LeetCodeDataset, PATSSEL reached 74.5% accuracy compared with 71.4% for regeneration and 64.4% for debugging under the same inference budget. We also trained routers that tried to predict the preferred strategy before seeing either repair candidate. They were weaker than verifier-based selection. This showed that the best strategy cannot always be inferred from the failure alone. In some cases, the system must explore alternative reasoning paths and evaluate what those paths actually produce.

Scaling Verifiable Experience for Agents

As my work moved from programs to agents, the bottleneck changed again. A learning algorithm can only improve an agent if the agent encounters tasks that expose behaviors it has not yet mastered. Existing agent benchmarks contain valuable tasks, but they are expensive to build and provide a limited amount of difficult, verifiable experience. During my internship with the Granite Model Training Team at IBM Research, I worked with Dr. Yousaf Shah on a question at this larger scale: how can we generate harder agentic tasks without producing examples that are inconsistent, unsolvable, or only harder in wording? This became *Agent Forest: Synthetically Hardening Complex RL Environments via a Multi-Agent Orchestration System*, currently under review at ICLR 2027 [4].

The key issue is that agentic difficulty lives in the interaction, not only in the instruction. A task can become harder because the agent must use more tools, branch on observations, remember information from earlier turns, recover missing information, or coordinate several goals. A language-only rewrite may sound more complex without changing any of these demands. Agent Forest therefore uses specialized ReAct-style hardening agents that inspect the current task, tools, reference solution, and environment state before modifying both the instruction and the executable trajectory. Different agents introduce tool expansion, branching, memory, partial observability, and compositionality. Each generated task is replayed in a fresh environment and checked for semantic consistency before it is accepted. Failed candidates are repaired or filtered rather than added to the dataset.

This project required working at the level of the full data and evaluation pipeline. Across TAU2-Retail, TAU2-Airline, and General-Agent, the accepted descendants expanded the original task collections to roughly 3.4 to 4.4 times their seed size. The resulting tasks were also harder for strong agents. Qwen 3.5 397B fell from 75.4% to 54.7% pass@1 on TAU2-Retail. Gemma 4 31B fell from 66.0% to 28.6% on TAU2-Airline and from 68.8% to 25.6% on General-Agent.

One result was particularly useful for understanding what the pipeline changed. On General-Agent, adding partial observability barely changed the reference trajectory length, from 43.80 to 43.82 tool calls, yet Gemma’s pass@1 fell from 43.9% to 24.4%. The solver itself made more tool calls after the transformation. The difficulty therefore did not come from simply making the task longer. It came from changing what information the agent had to recover and use. For me, this is the central value of synthetic agent data: it should not only create more examples, but also provide control over what kind of experience the model receives.

Research and Technical Breadth

Across these projects, I have worked across the full research and development cycle for language models. I have built large-scale synthetic datasets and data pipelines, designed controlled evaluations, trained language models with reinforcement learning, developed verifiable reward functions, and built systems that use execution feedback to improve or select model behavior. My work has also required training specialized components such as code debuggers and test generators, integrating models with databases and external tools, and evaluating systems across multiple model families and benchmarks.

More recently, my work has extended these skills to long-horizon agents. In Agent Forest, I worked with stateful tool-use environments, ReAct-style agents, large-model inference, synthetic task generation, environment inspection, executable trajectory generation, validation, and repair. The resulting pipeline could transform trusted seed tasks into larger collections of verified tasks while preserving consistency with the underlying environment. This required treating data generation as a systems problem, where models, tools, environment state, verification, and evaluation all had to work together.

Together, these projects have given me experience building ML systems from the research question through the final evaluation. I am comfortable working on problems that require both model development and the infrastructure around it, including post-training, reward design, synthetic data generation, code and agent evaluation, tool integration, and large-scale experimentation. I am particularly interested in roles where these pieces come together to improve foundation models and agents in real systems.

References

1. **LLM-Symbolic Integration for Robust Temporal Tabular Reasoning.** Atharv Kulkarni, Kushagra Dixit, Vivek Srikumar, Dan Roth, Vivek Gupta. The 63rd Annual Meeting of the Association for Computational Linguistics, ACL 2025 (Findings). [\[Code\]](#) [\[Publication\]](#) [\[Website\]](#)
2. **Reinforcing Code Generation: Improving Text-to-SQL with Execution-Based Learning.** Atharv Kulkarni, Vivek Srikumar. [\[Publication\]](#)
3. **Debug or Regenerate? Test-Guided Selection for LLM Code Repair.** Atharv Kulkarni, Archiki Prasad, Vivek Srikumar, Elias Stengel-Eskin, Hyunji Lee, Mohit Bansal. The 2026 Conference on Empirical Methods in Natural Language Processing (Findings). [\[Publication\]](#) [\[Code\]](#)
4. **Agent Forest: Synthetically Hardening Complex RL Environments via a Multi-Agent Orchestration System.** Atharv Kulkarni, Santosh Borse, Hajar Emami Gohari, Mohit Bansal, Rameswar Panda, Yousaf Shah. Under review at ICLR 2027.